AMERICAN UNIVERSITY OF SCIENCE & TECHNOLOGY

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER AND COMMUNICATIONS ENGINEERING

DEPARTMENT OF MECHATRONICS ENGINEERING

CONTROL SYSTEMS LAB

CCE 405L

Project Report

Elevator Control Systems

Prepared by

Maher Alaa Deen

Khaled Bayan

Rami Hussein

Amir Jibawi

Presented to

Gaby Abou Haidar. Ph.D.

July 1, 2022

Achrafieh Campus

ABSTRACT

This abstract presents an elevator control system that utilizes a dual Proportional-Integral-Derivative (PID) controller to regulate the position and velocity of a DC motor. The primary objective of the system is to ensure smooth and accurate elevator movement, enhancing user experience and safety.

The elevator control system employs a DC motor to control the motion of the elevator car. To achieve precise control, a dual PID controller is implemented. The position PID controller is responsible for accurately positioning the elevator car at desired levels, while the velocity PID controller regulates the speed of the motor to ensure smooth and controlled movement.

The position PID controller receives the desired position setpoint and the current position feedback from sensors or encoders. It calculates the error between the setpoint and the actual position, and then applies appropriate control signals to the motor. The integral and derivative terms help in reducing steady-state errors and smoothing out rapid changes in position, respectively.

Similarly, the velocity PID controller receives the desired velocity setpoint and the current velocity feedback from sensors. It computes the error between the setpoint and the actual velocity, and generates control signals to adjust the motor speed accordingly. The integral term helps maintain constant velocity, while the derivative term aids in damping oscillations and reducing overshoot.

By combining position and velocity control using dual PID controllers, the elevator control system achieves precise and efficient control over the DC motor. The system ensures accurate positioning of the elevator car at desired floors, while maintaining a smooth and controlled velocity throughout the journey.

The implementation of the dual PID controller in the elevator control system enhances the overall performance, reliability, and safety of the elevator system. It minimizes position and velocity errors, reduces jerky movements, and provides a comfortable and secure user experience.

List of Figures

Figure 1. Elevator Design 2
Figure 2. Elevator Cell Schematic 5
Figure 3. Elevator Cell Assembly 3D 5
Figure 4. Elevator Body 6
Figure 5. Elevator Body 3D 7
Figure 6. Elevator Full Assembly Design 8
Figure 7. Elevator Implementation 9
Figure 8. H-Bridge Schematic 10
Figure 9. H-Bridge PCB Layout 11
Figure 10. H-Bridge 3D View 12
Figure 11. H-Bridge PCB View 2 13
Figure 12. Electrical Design Implementation 14
Figure 13. Transfer Function Step Response using PID Tuner 15
Figure 14. Position PID Controller Root Locus 16
Figure 15. Position PID Bode Plot 16
Figure 16. Position PID System Step Response 17
Figure 17. Dual PID Step Response 18
Figure 18. Dual PID Bode Plot 18
Figure 19. Dual PID Root Locus 19
Figure 20. Dual PID Step Response 19
Figure 21. Elevator Control System Block Diagram 20

LIST OF TABLES

Table 1. Position PID Controller Parameters.....15
Table 2. Position PID Controller Performance and Robustness.....15
Table 3. PID Controller Parameters.....20
Table 4. Dual PID Performance and Robustness.....20

Table of Contents

I. OBJECTIVES **1**
II. BACKGROUND THEORY **1**
III. REQUIRED COMPONENTS **4**
IV. SCHEMATIC REPRESENTATION **4**
V. PROCEDURE **20**
VI. CODE **21**
VII. CONCLUSION **24**

I. Objectives

1. To explain the concept of a PID controller.
2. Design an Elevator using Autodesk Inventor.
3. Design, test and implement a dual PID controller for an elevator system to control its position and velocity.

II. Background Theory

Elevators, also known as lifts, are vertical transportation systems designed to transport people or goods between different floors or levels of a building. Elevators play a crucial role in modern architecture and have become an integral part of high-rise buildings, commercial complexes, residential towers, and other structures. The operation and control of elevators involve various engineering principles and technologies to ensure safe, efficient, and reliable transportation.

Basic Elevator Components:

An elevator system consists of several key components:

1. Cab/Car: The elevator car is the enclosed platform that carries passengers or goods. It moves vertically along guide rails, providing transportation between different floors.
2. Hoistway/Shaft: The hoistway, also called the elevator shaft, is the vertical space in which the elevator car travels. It is constructed with sturdy walls and contains the guide rails, counterweights, and other essential components.
3. Counterweights: Counterweights are designed to offset the weight of the elevator car and its occupants. They help reduce the overall energy required for vertical movement and enhance the system's efficiency.
4. Traction System or Hydraulic System: Elevators typically employ either a traction system or a hydraulic system for vertical motion. The traction system uses ropes or belts connected to an electric motor, whereas the hydraulic system relies on hydraulic fluid and a piston arrangement to move the elevator car.
5. Control System: The control system manages the operation of the elevator, including starting, stopping, and speed regulation. It monitors various sensors,

processes user inputs, and ensures the safety and smooth functioning of the elevator.

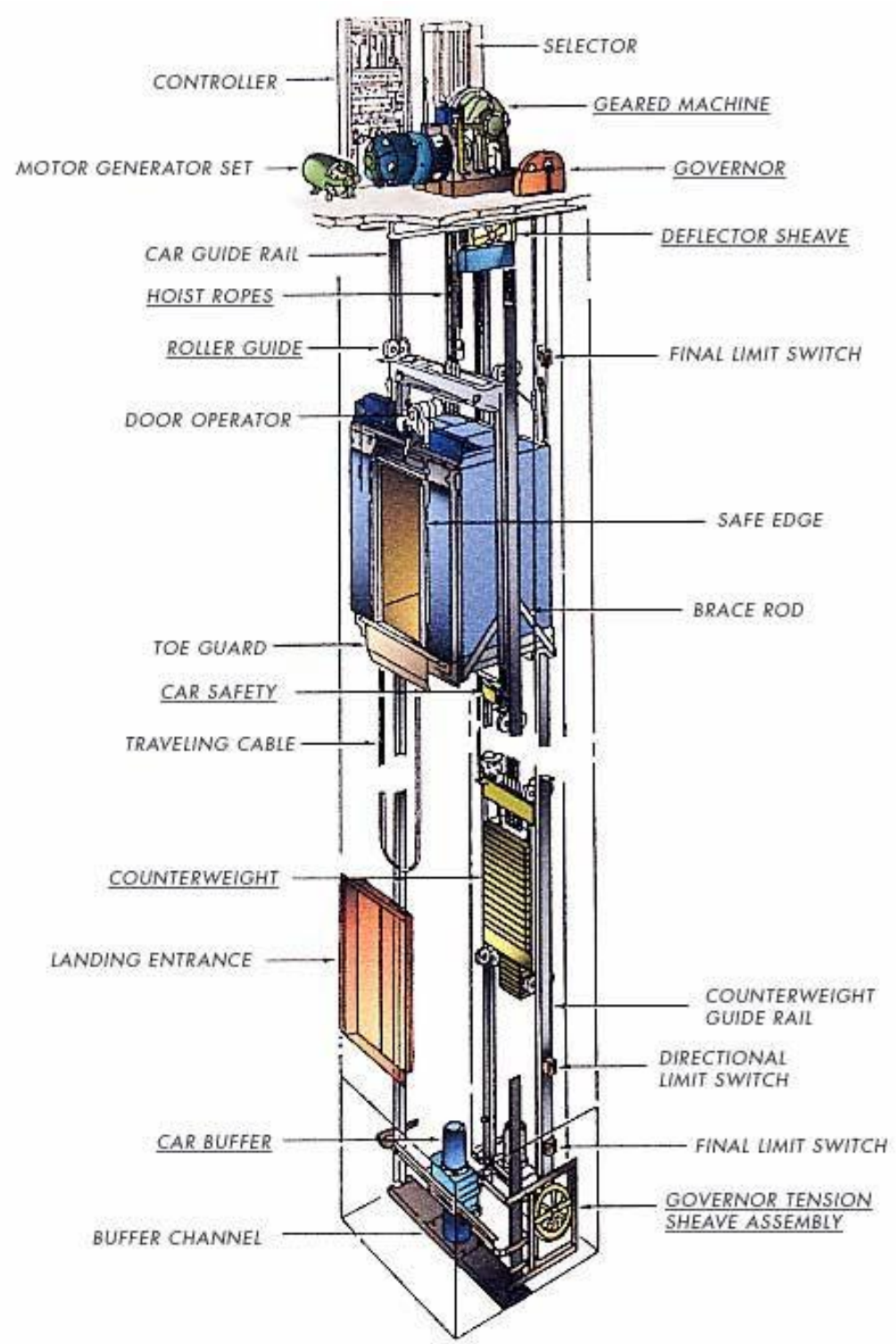

Figure 1. Elevator Design

Elevator Control Systems:

Elevator control systems are responsible for regulating the movement of the elevator car, ensuring precise positioning, smooth acceleration and deceleration, and efficient operation. Various control strategies can be implemented, including:

1. Simple Relay Control: This basic control system uses electromechanical relays to manage the elevator's operation. It involves limit switches to detect floor positions, relays for starting and stopping the motor, and basic logic circuits for floor selection.
2. Microprocessor-based Control: Modern elevator control systems often utilize microprocessors or programmable logic controllers (PLCs) for advanced functionality and flexibility. These systems can incorporate more sophisticated algorithms, monitoring sensors, and safety features.

PID Control in Elevator Systems:

Proportional-Integral-Derivative (PID) control is a popular control technique widely employed in elevator systems. The PID controller continuously calculates an error signal, which represents the difference between the desired position or speed and the actual position or speed of the elevator. The controller then adjusts the control inputs, such as motor voltage or hydraulic pressure, based on the proportional, integral, and derivative terms to minimize the error and achieve the desired performance.

The position PID controller focuses on accurately controlling the elevator's stopping position at each floor, while the velocity PID controller regulates the speed and acceleration/deceleration profiles to ensure smooth and comfortable motion between floors.

Benefits of PID Control in Elevators:

PID control offers several advantages in elevator systems:

1. Precise Positioning: PID control helps achieve accurate and repeatable floor positioning, minimizing deviations and providing a comfortable boarding and disembarking experience for passengers.
2. Smooth Motion: By regulating velocity and acceleration profiles, PID control ensures smooth and jerk-free elevator movement, reducing passenger discomfort and potential accidents.

3. Robust Performance: PID control can adapt to varying loads, changing passenger capacities, and external disturbances, maintaining stable and reliable operation under different conditions.
4. Easy Tuning: PID control parameters can be adjusted and tuned to optimize elevator performance based on specific requirements, allowing for customization and fine-tuning as needed.

III. Required Components

1. Arduino Nano
2. Ultrasonic Sensor
3. Photoelectric encoder HC-020K
4. Resistors: 1KΩ x6
5. TIP120 x4
6. BC547B x4
7. 1N914B x4
8. Limit Switch x2
9. Terminal Block I4 x2
10. Terminal Block I2 x2
11. DC Motor

IV. Schematic Representation

1. <u>Mechanical Design:</u>

The elevator was constructed using Inventor software, in which we created an outer elevator structure resembling a cell shape. Additionally, we incorporated a sliding cell that moves along three rods, utilizing linear bearings for its smooth motion. In our elevator design, we implemented a double-sided pulley mechanism to facilitate the upward and downward movement. This pulley system is connected to a DC motor of our choice. Furthermore, we incorporated a counterweight system that works in tandem with the pulley, ensuring balanced and efficient operation of the elevator.

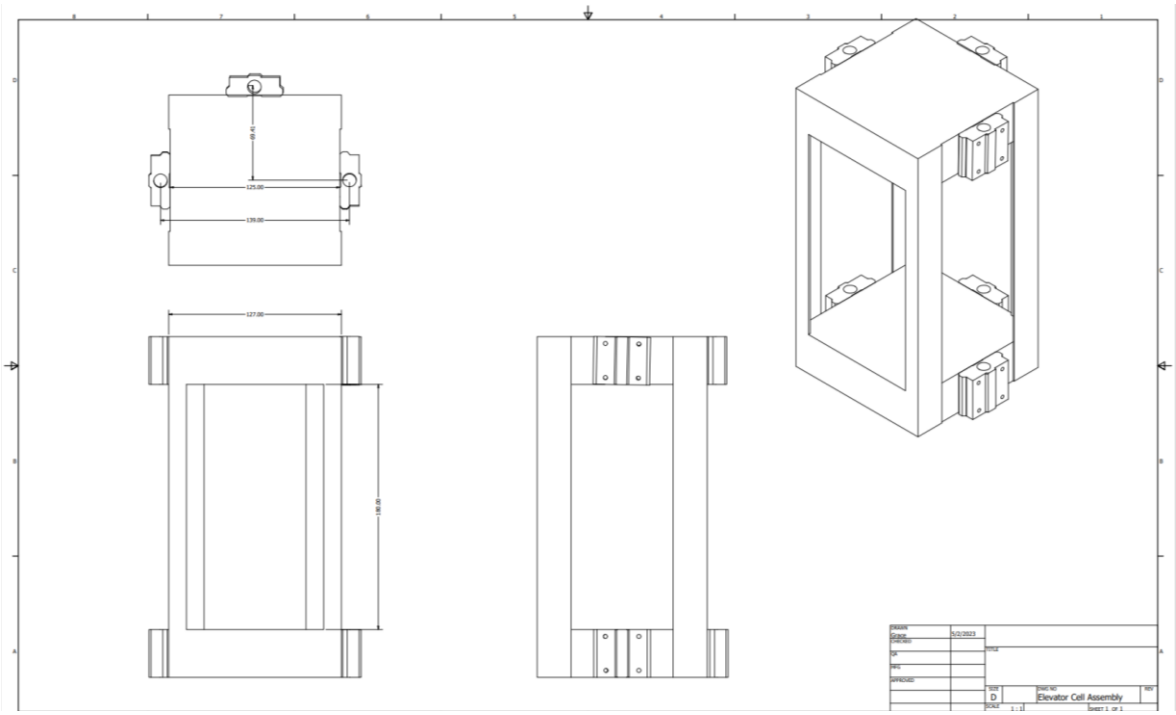

Figure 2. Elevator Cell Schematic

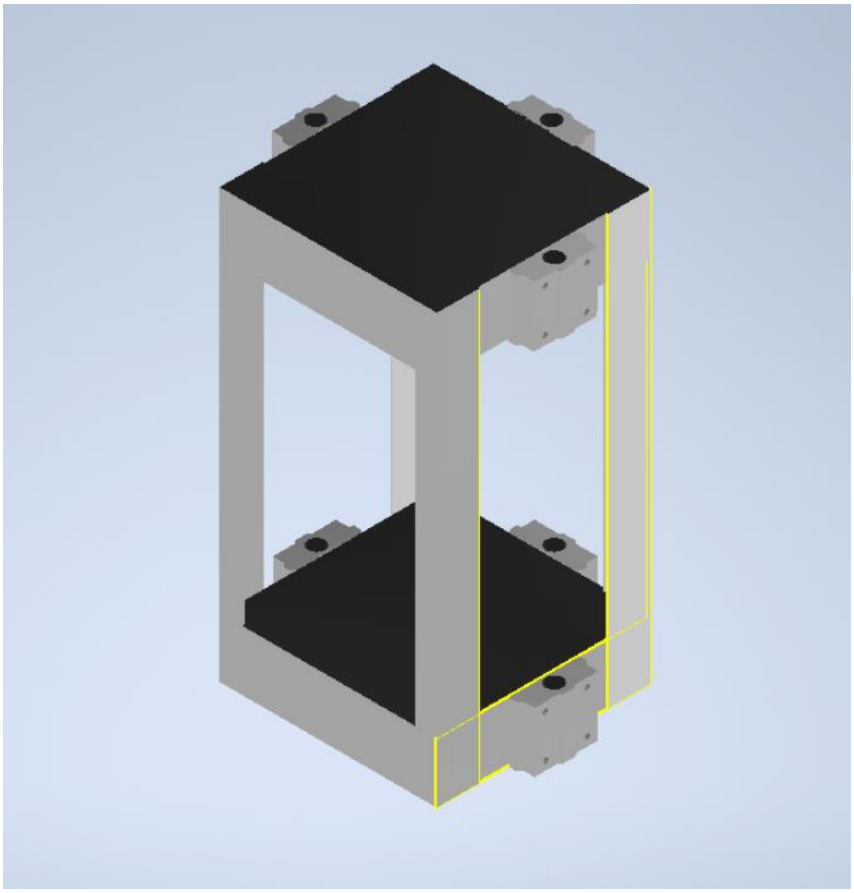

Figure 3. Elevator Cell Assembly 3D

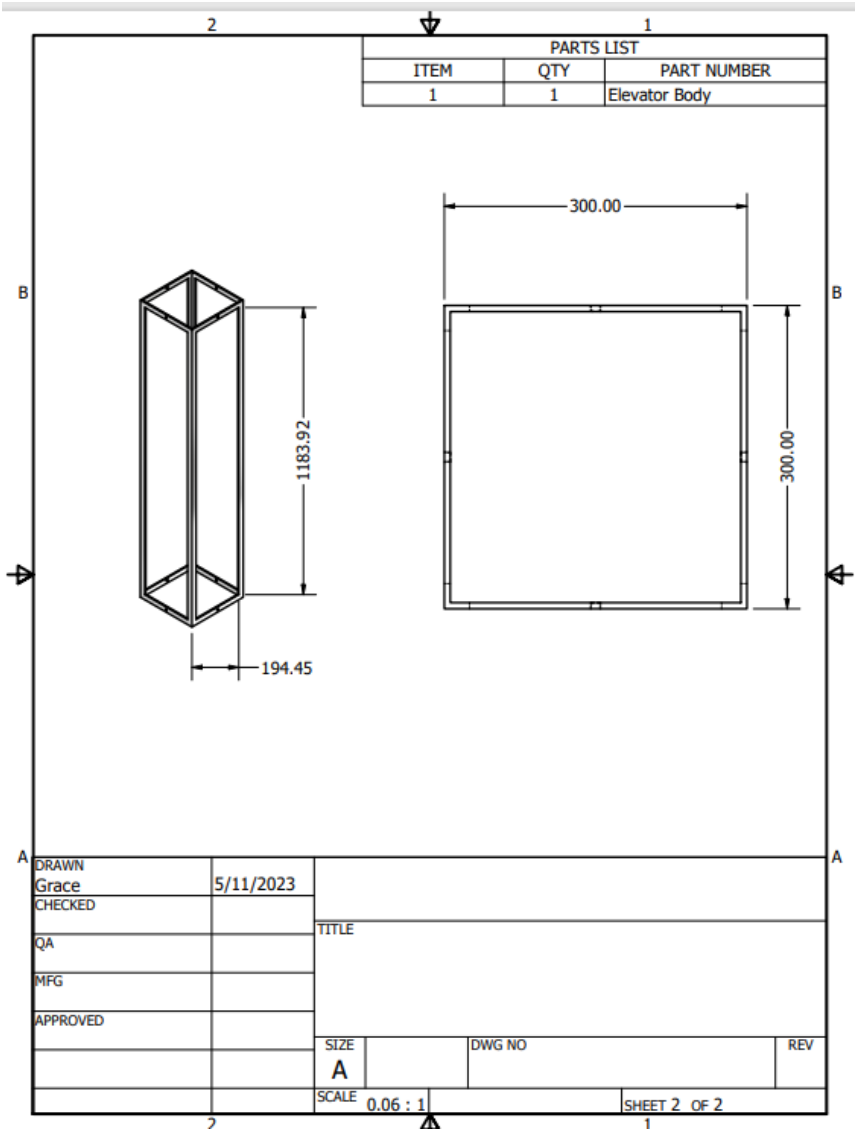

Figure 4. Elevator Body

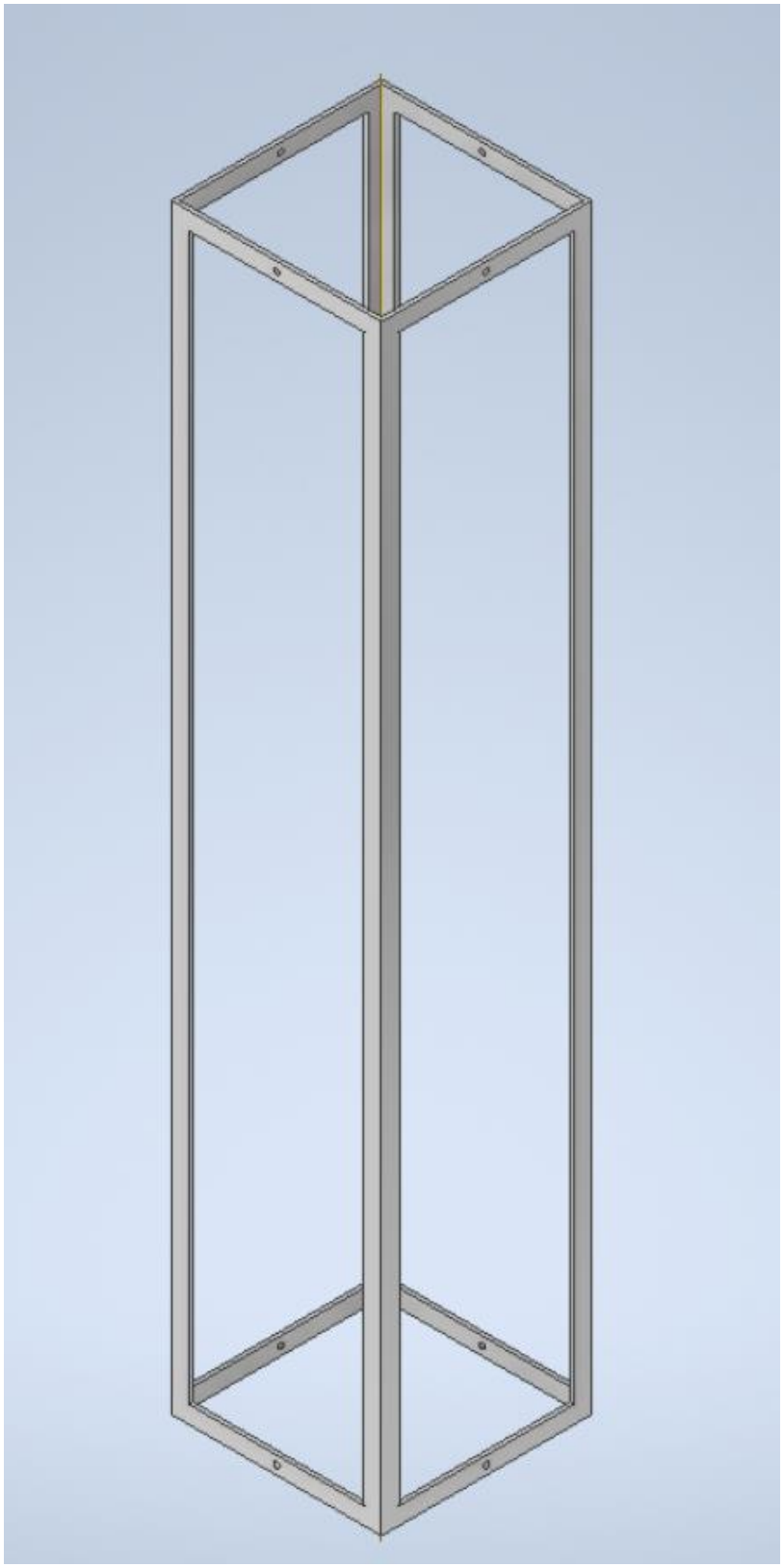

Figure 5. Elevator Body 3D

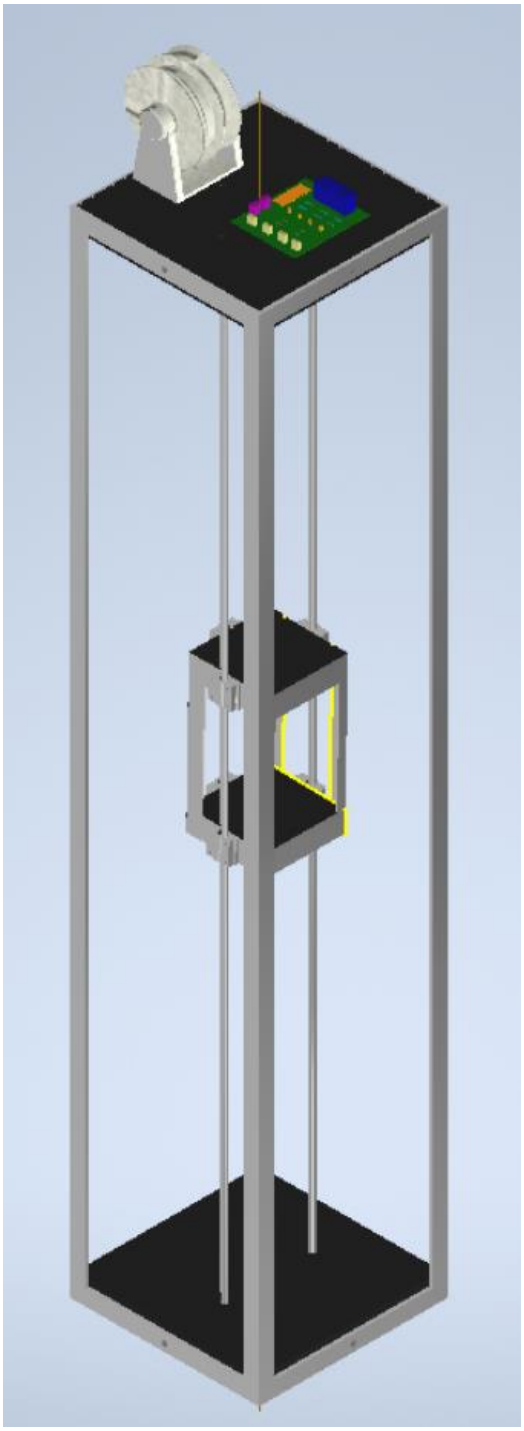

Figure 6. Elevator Full Assembly Design

Figure 7. Elevator Implementation

2. <u>Electrical Design:</u>

Using Altium Designer, we developed an H-Bridge configuration specifically designed to control the DC motor in both clockwise (cw) and counterclockwise (ccw) directions.

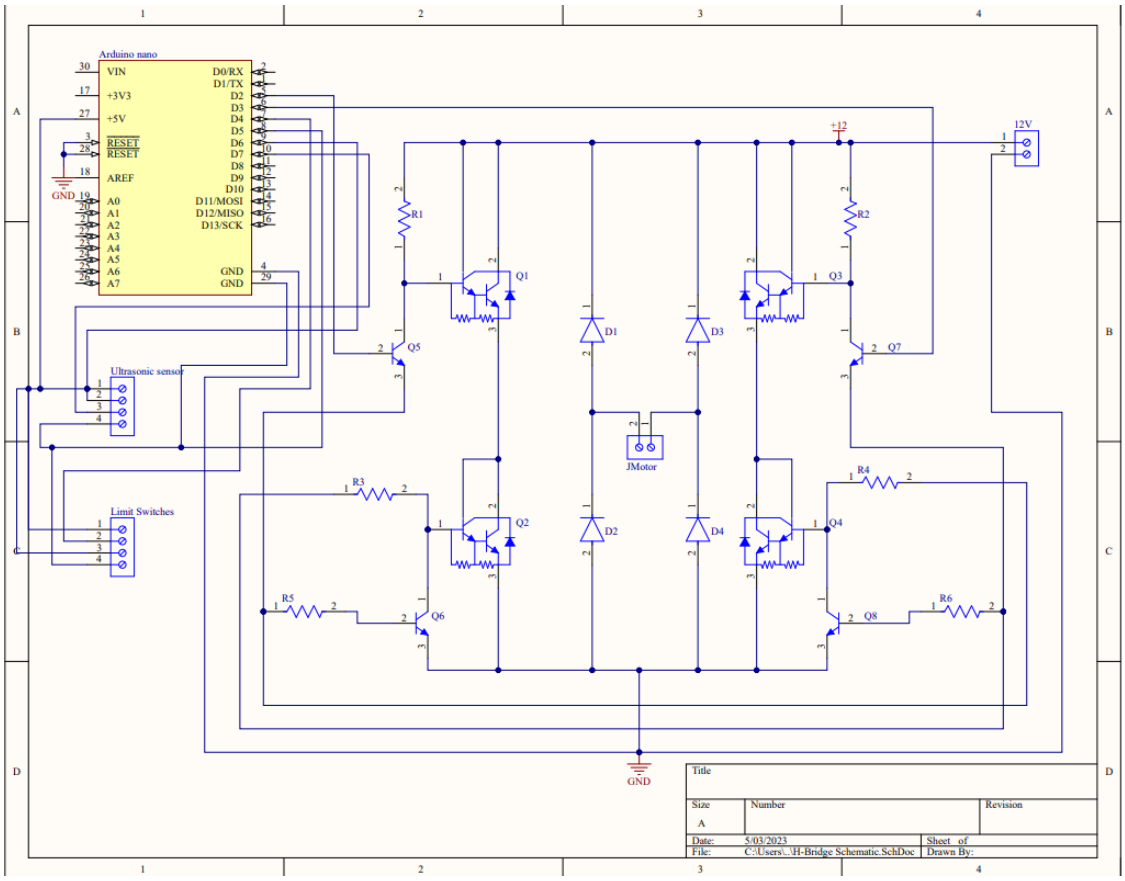

Figure 8. H-Bridge Schematic

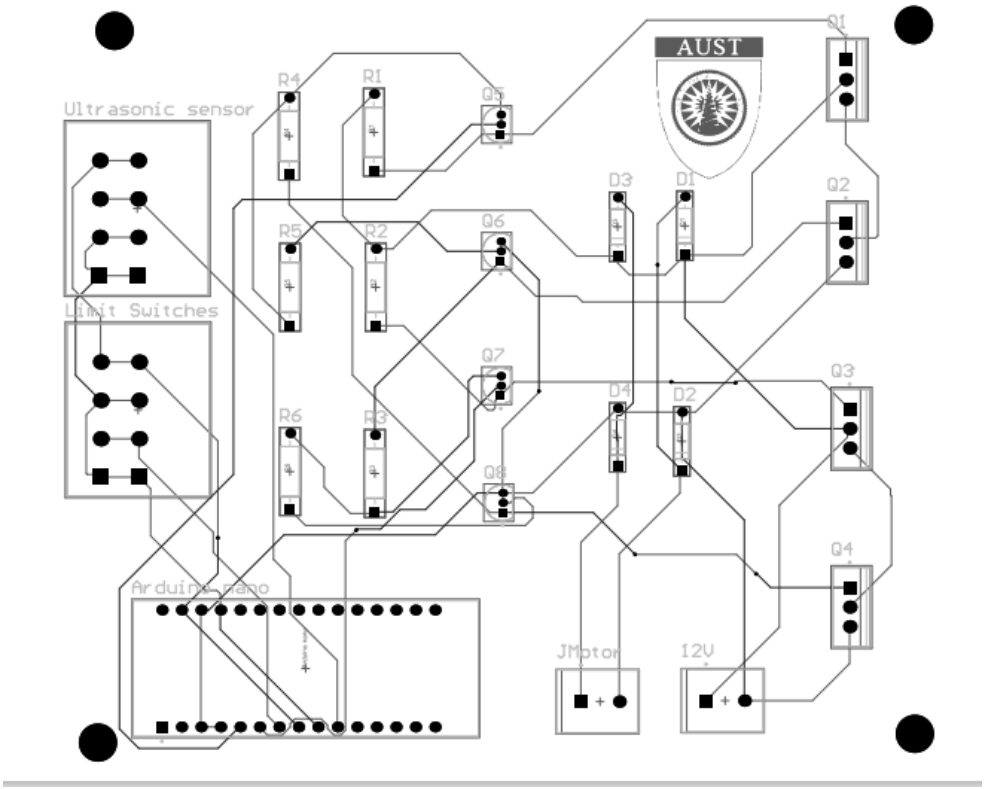

Figure 9. H-Bridge PCB Layout

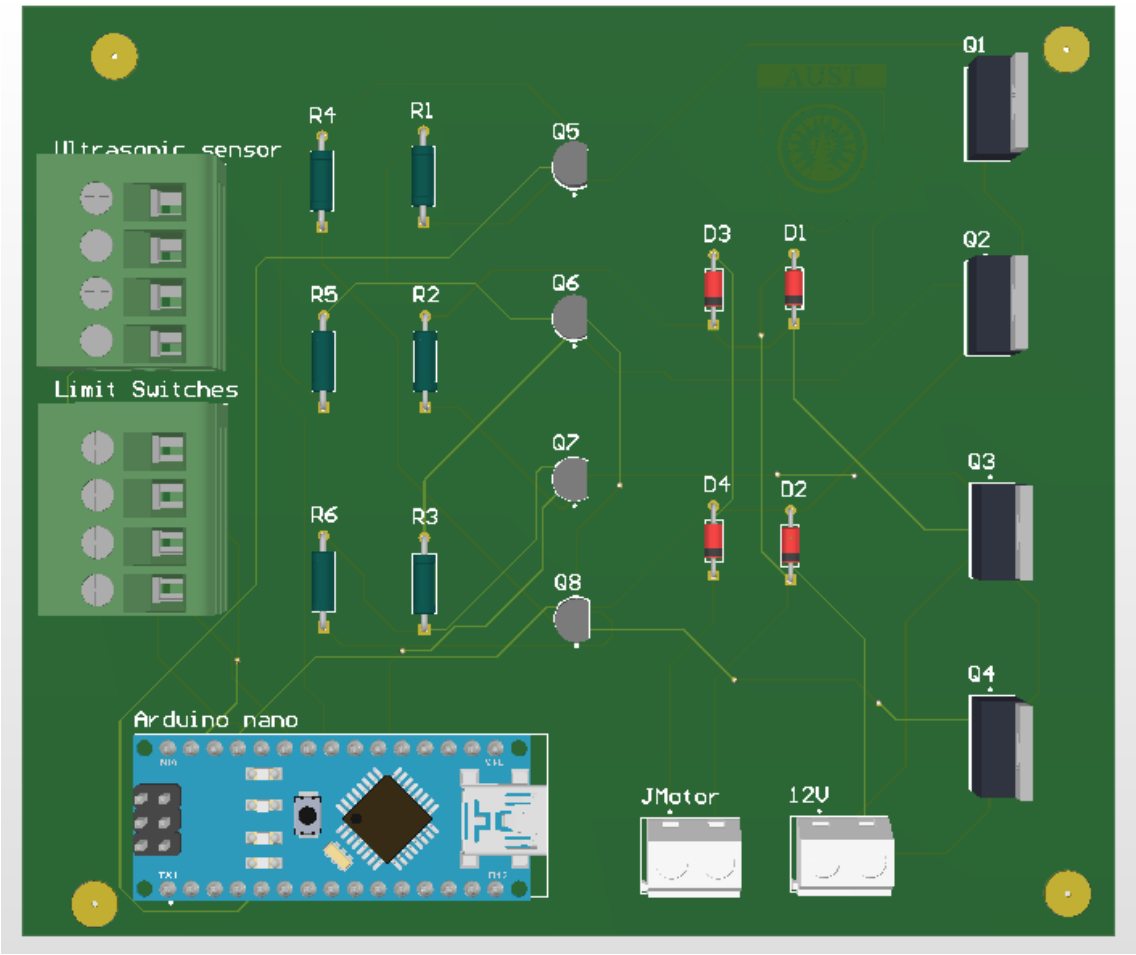

Figure 10. H-Bridge 3D View

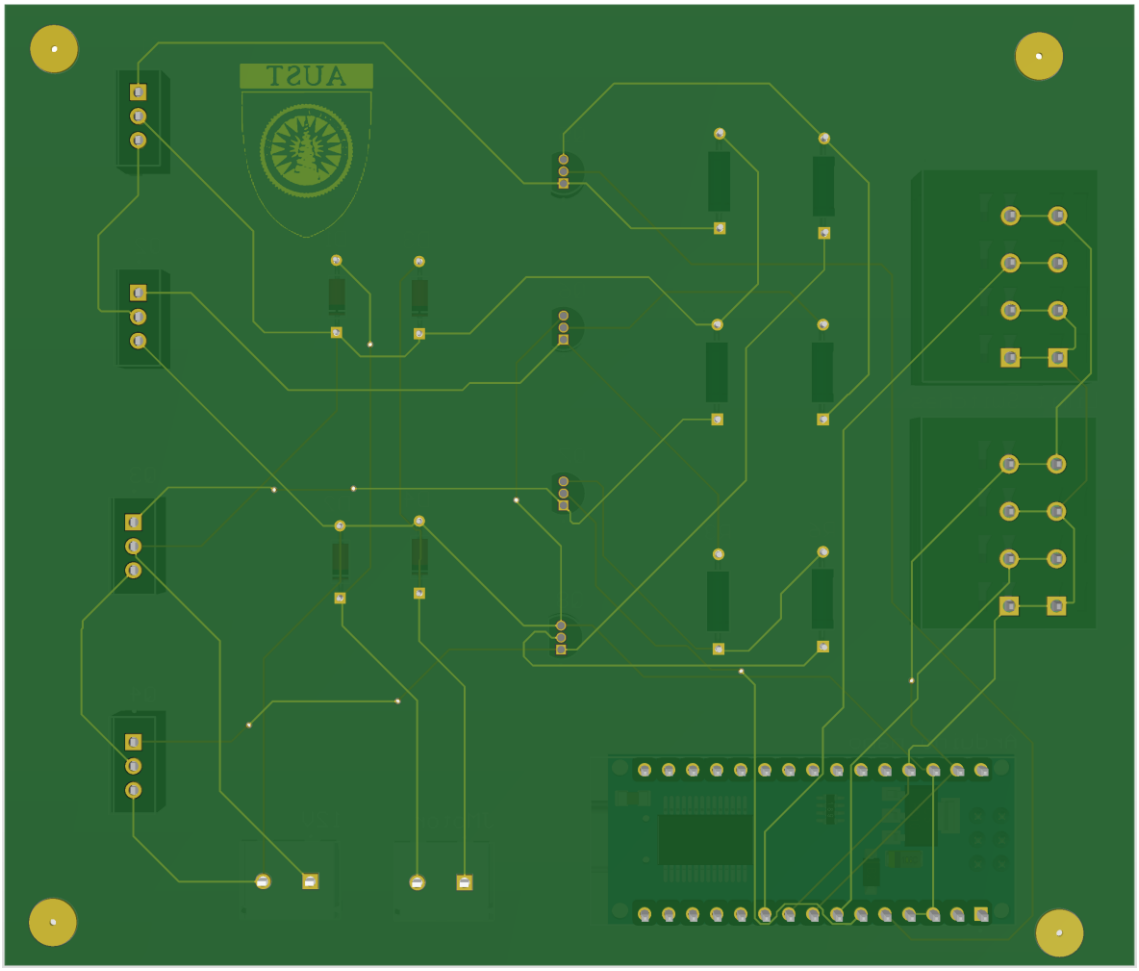

Figure 11. H-Bridge PCB View 2

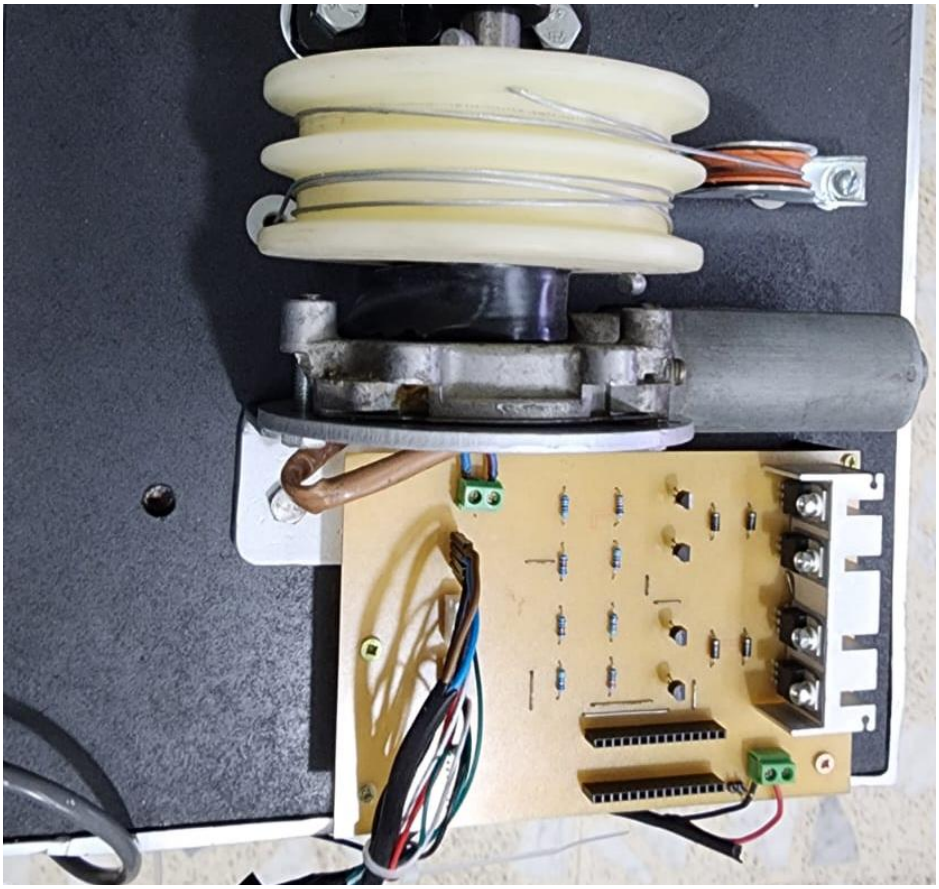

Figure 12. Electrical Design Implementation

3. <u>Control design:</u>

   <u>Position PID Tuning:</u>

   The DC Motor Transfer function found using MATLAB's System identification toolbox is the following:

$$\frac{\theta_m(s)}{E_a(s)} = \frac{72.59}{s(s+1.667)}$$

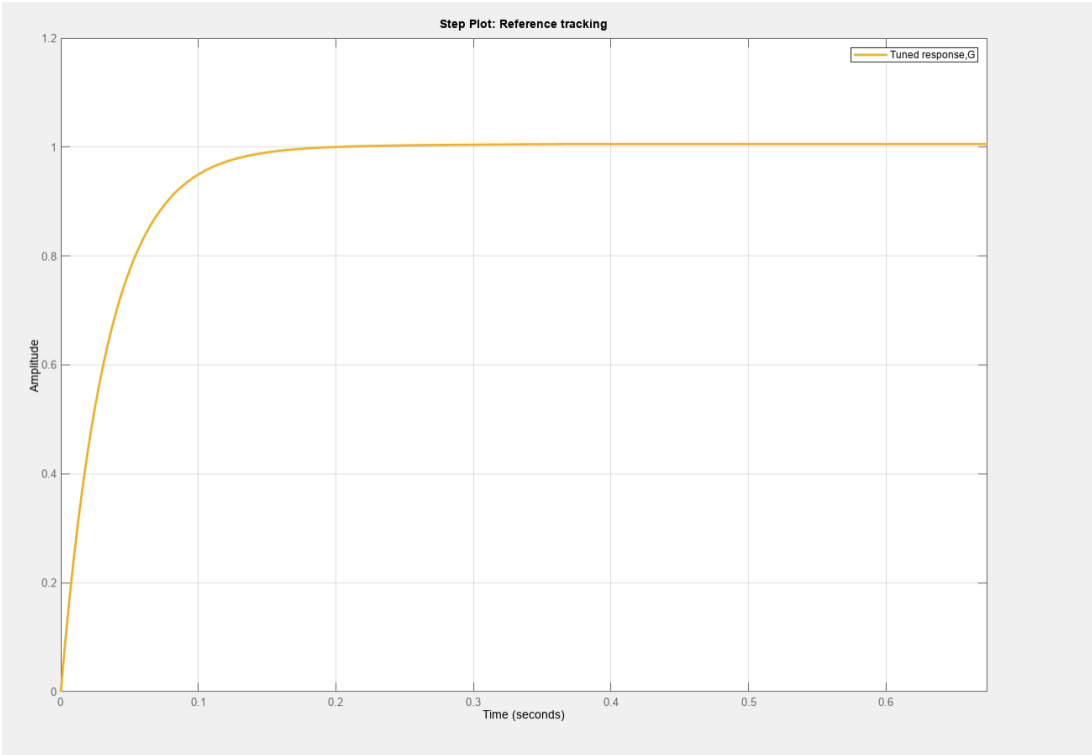

Figure 13. Transfer Function Step Response using PID Tuner

The tuned parameters for the Position PID Controller are listed in the Tables below:

Table 1. Position PID Controller Parameters

	Tuned
Kp	0.68181
Ki	0.28391
Kd	0.40933

Table 2. Position PID Controller Performance and Robustness

	Tuned
Rise Time	0.25 Seconds
Settling Time	0.25 Seconds
Overshoot	0.29%
Peak	1.01
Gain margin	-inf dB @ 0 rad/s
Phase margin	90 deg @ 29.7 rad/s
Closed-loop Stability	Stable

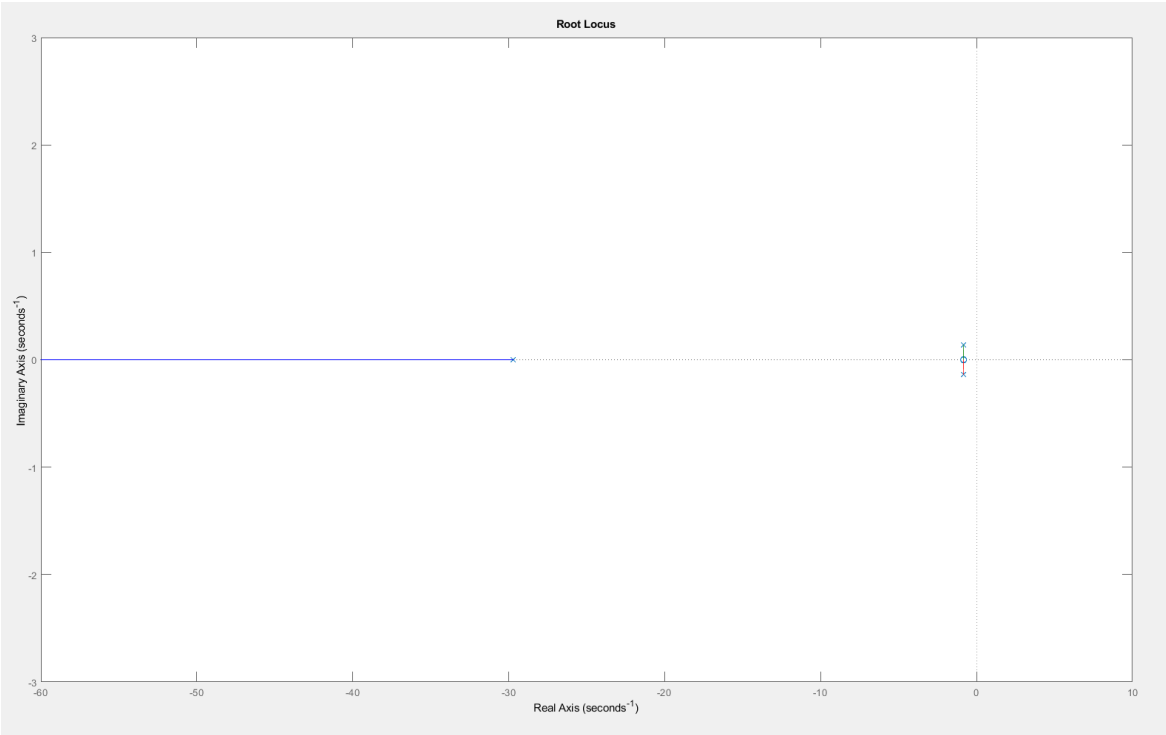

Figure 14. Position PID Controller Root Locus

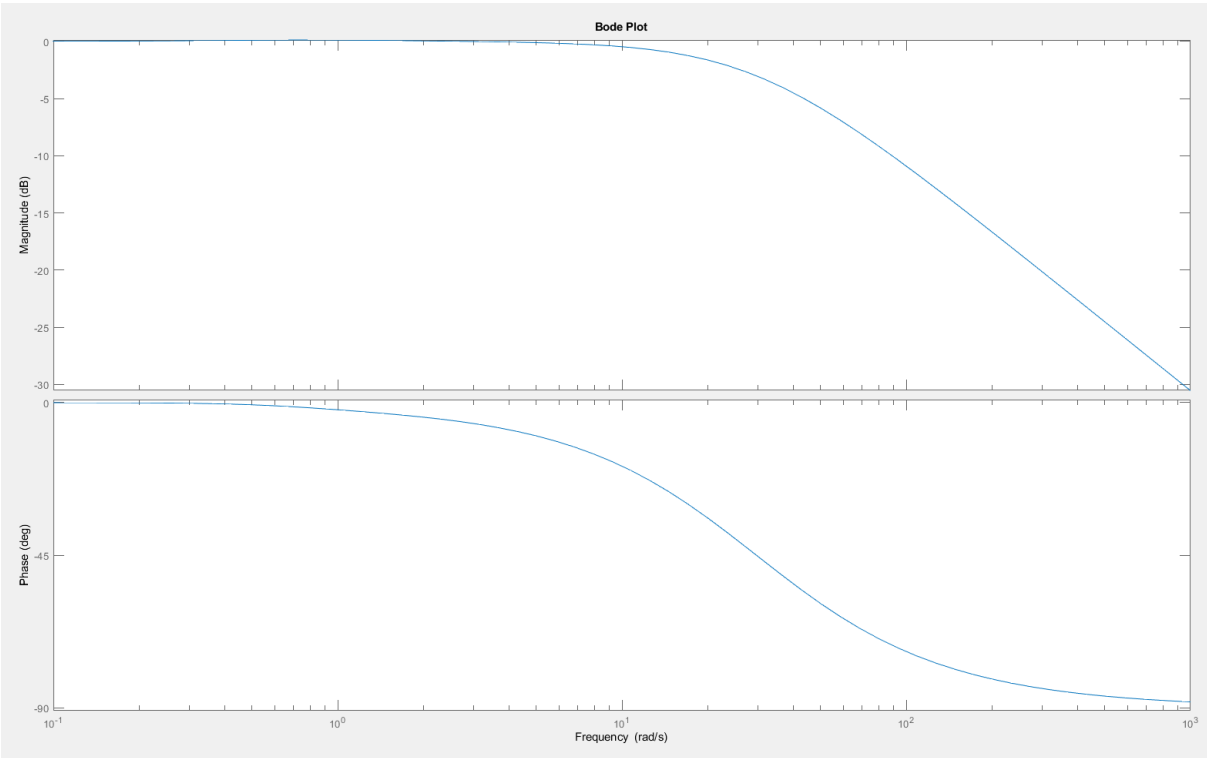

Figure 15. Position PID Bode Plot

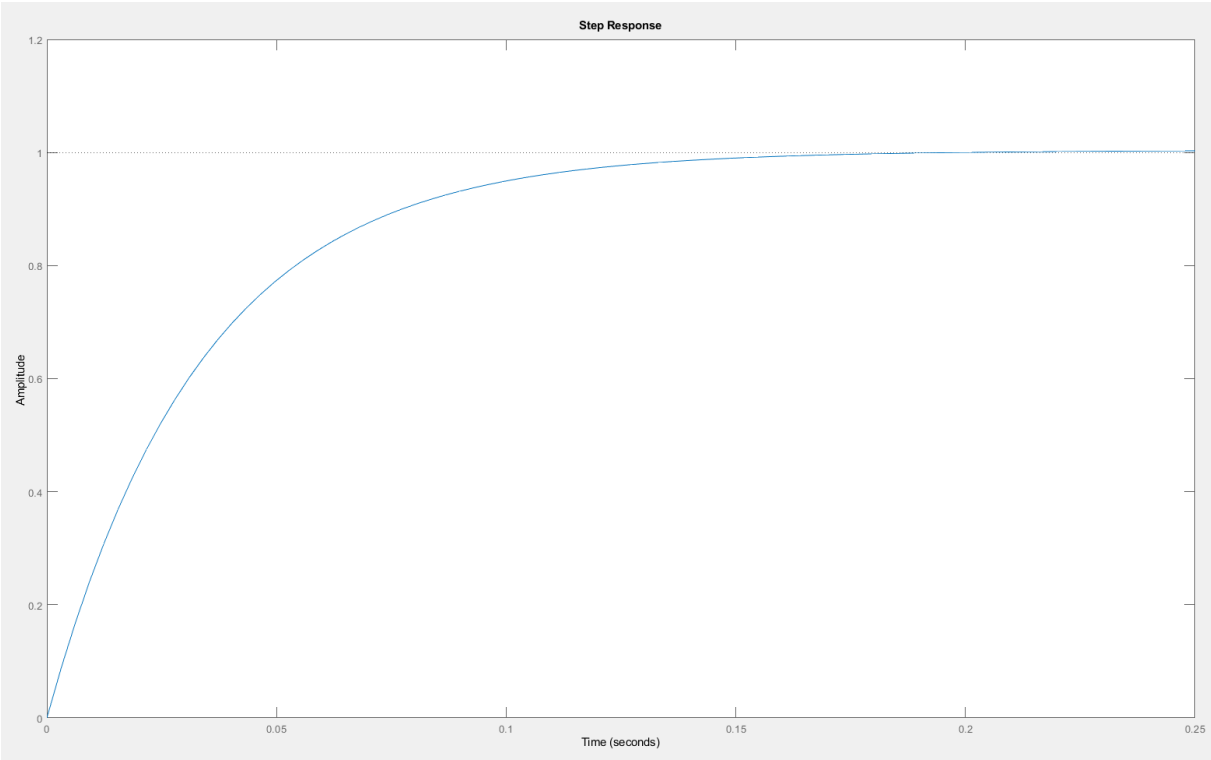

Figure 16. Position PID System Step Response

<u>Velocity PID Tuning:</u>

To control the Elevator's velocity the following transfer function is presented:

$$\frac{s\theta_m(s)}{E_a(s)} = \frac{72.59s}{s(s+1.669)} = \frac{72.59}{s+1.669}$$

Since $V(t) = \frac{dr(t)}{dt}$ in the time domain, so the velocity in the frequency domain will become $V(s) = sR(s)$ where R represents the position vector of the elevator cell.

The 2 PID controllers are connected in parallel:

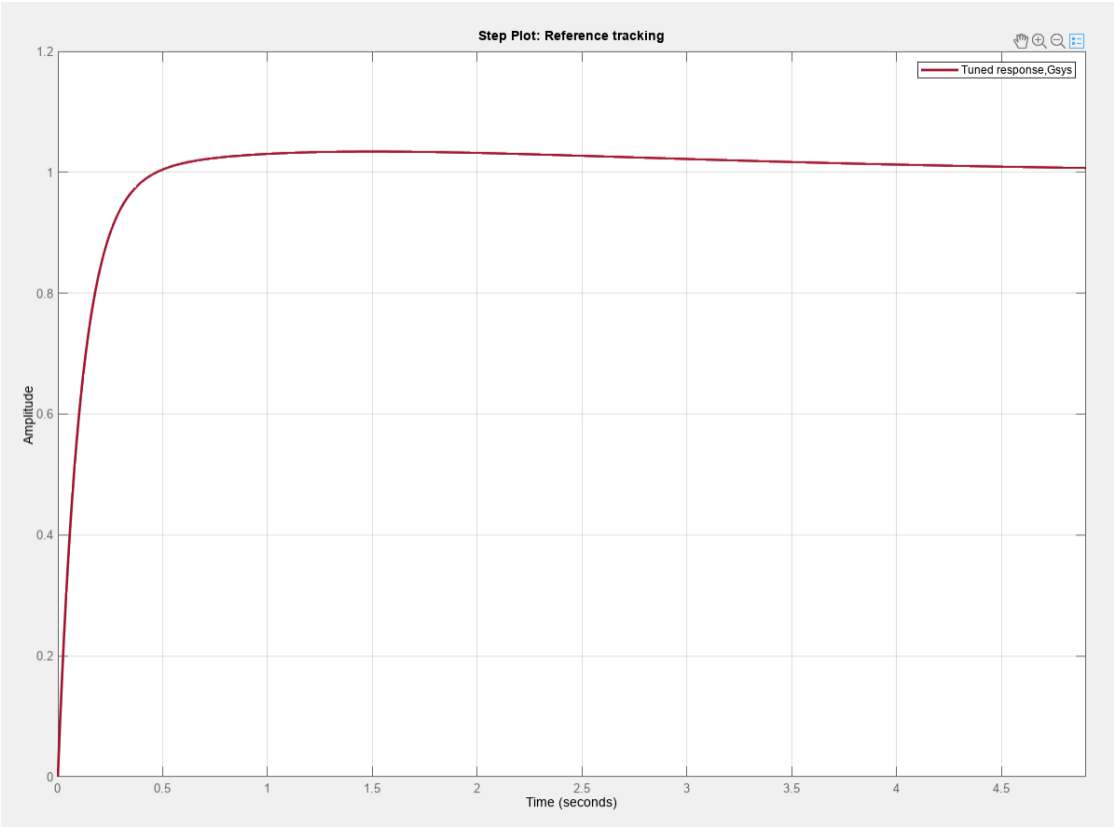

Figure 17. Dual PID Step Response

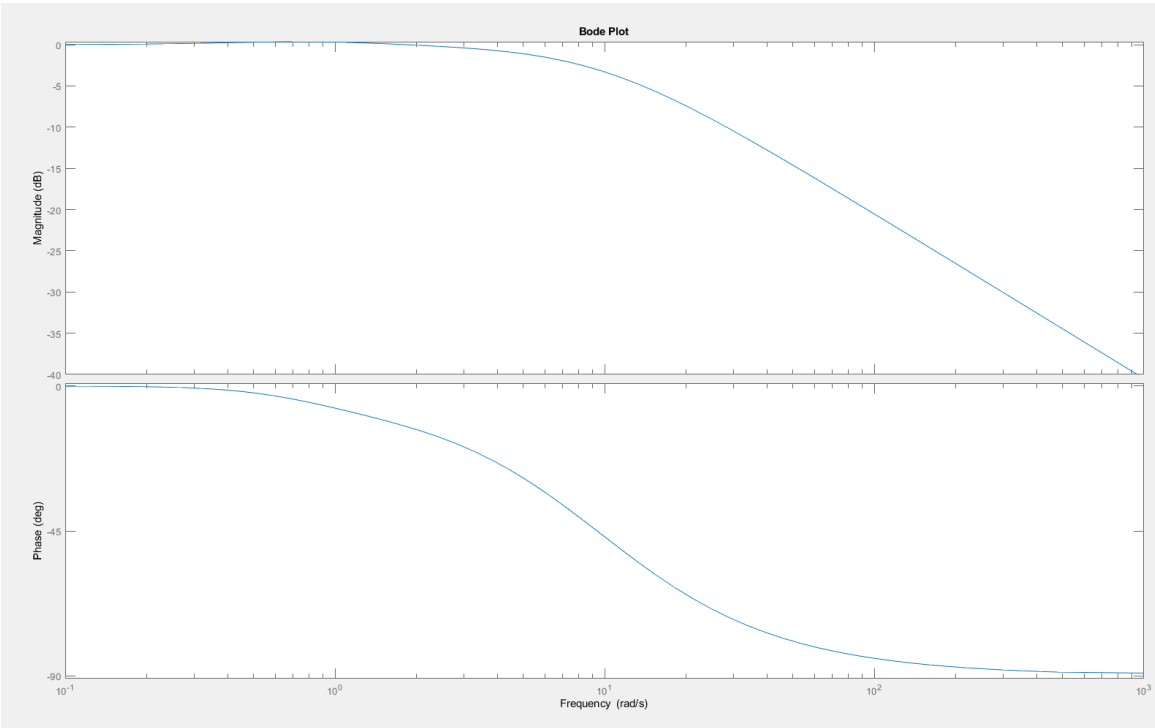

Figure 18. Dual PID Bode Plot

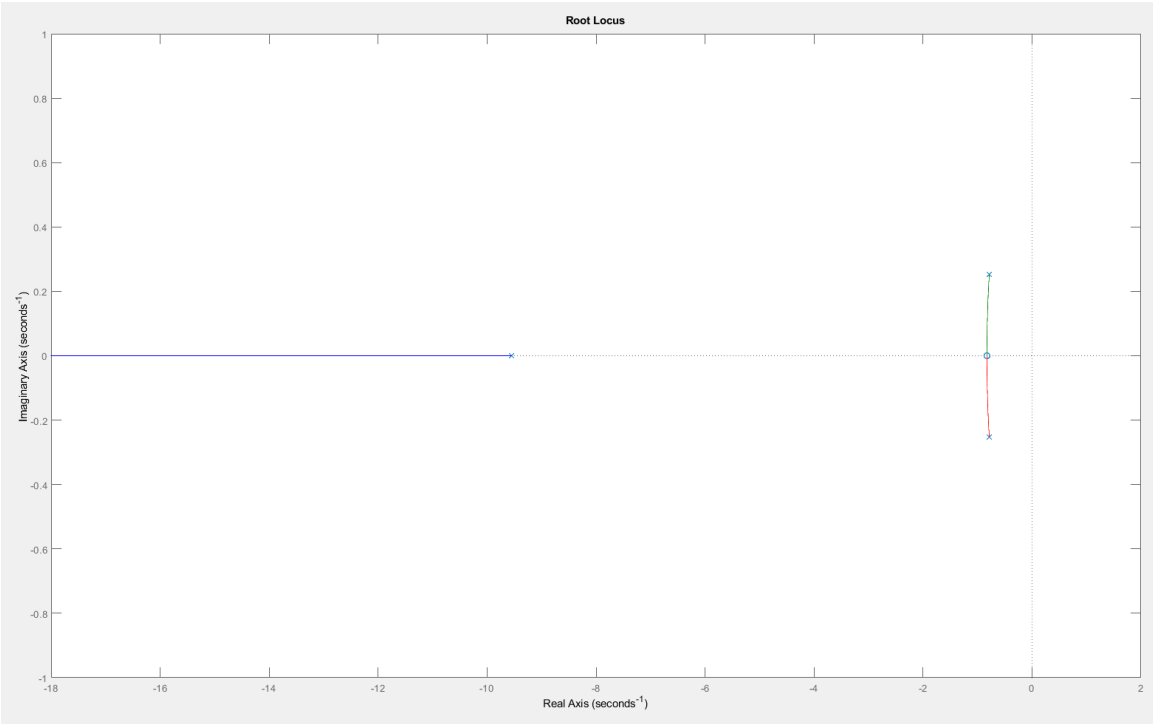

Figure 19. Dual PID Root Locus

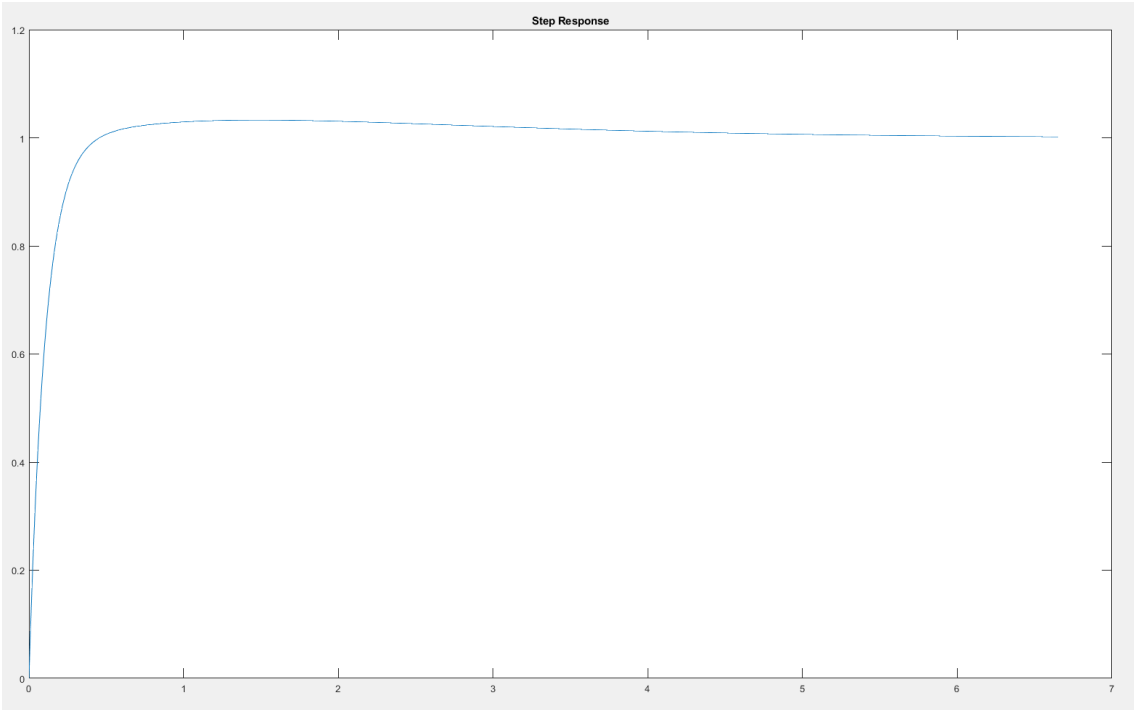

Figure 20. Dual PID Step Response

Table 3. PID Controller Parameters

	Tuned
Kp	14.9465
Ki	6.176
Kd	9.0429

Table 4. Dual PID Performance and Robustness

	Tuned
Rise Time	1.48 Seconds
Settling Time	6.65 Seconds
Overshoot	3.28%
Peak	1.03
Gain Margin	-Inf dB @ 0 rad/s
Phase Margin	90 deg @ 8.97 rad/s
Closed-loop Stability	Stable

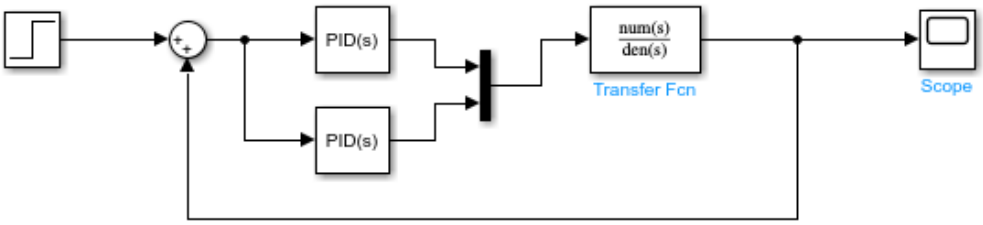

Figure 21. Elevator Control System Block Diagram

V. Procedure

1- Connect the DC motor to a potentiometer in order to control the supplied voltage to the motor.
2- Connect an encoder in order to measure the motor's speed and position.
3- Send the collected data serially to the MATLAB in order to use the systems identification toolbox to identify the transfer function of the system.
4- After estimating the transfer function, tune the position and velocity dual PID using PID tuner on MATLAB and plot the step response, bode plot and root locus of the system.

5- Write the corresponding arduino code in order to control the elevator position and speed.

VI. Code

MATLAB Code for the required plots:

```
% Define the transfer function
num = [1];  % numerator coefficients
den = [1, 1.667, 0];  % denominator coefficients
Gsys = tf(num, den);  % create the transfer function object

% Define PID gains
Kp1 = 0.68181;  % Proportional gain for PID1
Ki1 = 0.28391;  % Integral gain for PID1
Kd1 = 0.40933;  % Derivative gain for PID1

Kp2 = 14.9465;  % Proportional gain for PID2
Ki2 = 6.176;  % Integral gain for PID2
Kd2 = 9.0429;  % Derivative gain for PID2

% Create dual PID controller
C1 = pid(Kp1, Ki1, Kd1);
C2 = pid(Kp2, Ki2, Kd2);
C = parallel(C1, C2);

% Connect dual PID controller to the transfer function
sys = feedback(C * G, 1);

% Step response
[y_step, t_step] = step(sys);
figure;
plot(t_step, y_step);
title('Step Response');

% Impulse response
[y_impulse, t_impulse] = impulse(sys);
figure;
plot(t_impulse, y_impulse);
title('Impulse Response');

% Compute rise time, settling time, and percent overshoot for step response
Ts = t_step(find(abs(y_step - 1) < 0.02, 1, 'last')) - t_step(find(abs(y_step
- 1) > 0.02, 1));
OS = max(y_step) - 1;
TP = t_step(find(y_step == max(y_step)));

fprintf('Rise Time (Step Response): %.2f\n', TP);
fprintf('Settling Time (Step Response): %.2f\n', Ts);
fprintf('Percent Overshoot (Step Response): %.2f%%\n', OS * 100);

% Root locus
figure;
rlocus(sys);
title('Root Locus');
```

```
% Bode plot
figure;
bode(sys);
title('Bode Plot');
```

Arduino Code:

```
#include <PID_v2.h>

// Define motor pins
const int motorPin1 = 9;
const int motorPin2 = 10;
const int trigPin = 2;
const int echoPin = 3;
const int encoderPin = 7;
const int Limit1 = 4;
const int Limit2 = 5;
unsigned int rpm;
unsigned int pulses_per_turn = 20;
int diameter = 10;
double rotation;
int count;
long duration;
long distance;
// Define position PID constants
double Kp_position = 0.68181;  // Proportional gain for position
double Ki_position = 0.28391;  // Integral gain for position
double Kd_position = 0.40933;  // Derivative gain for position

// Define speed PID constants
double Kp_speed = 14.9465;     // Proportional gain for speed
double Ki_speed = 6.176;     // Integral gain for speed
double Kd_speed = 9.0429;    // Derivative gain for speed

// Define position and speed setpoints and inputs
double positionSetpoint = 1500;  // Desired position
double positionInput = 0.0;      // Current position
double speedSetpoint = 0.3;      // Desired speed
double speedInput = 0.1;         // Current speed

// Define position and speed PID controller objects
PID positionPID(&positionInput, &positionOutput, &positionSetpoint, Kp_position, Ki_position,
Kd_position, DIRECT);
PID speedPID(&speedInput, &speedOutput, &speedSetpoint, Kp_speed, Ki_speed, Kd_speed, DIRECT);

// Define PID output variables
double positionOutput = 0.0;
double speedOutput = 0.0;

// Define motor control limits
const int motorMin = 0;    // Minimum motor speed
const int motorMax = 255;  // Maximum motor speed
```

```
void setup() {
  Serial.begin(9600);
  // Initialize motor pin as an output
  pinMode(motorPin1, OUTPUT);
  pinMode(motorPin2, OUTPUT);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(encoderPin, INPUT);
  pinMode(Limit1, INPUT);
  pinMode(Limit2, INPUT);

  // Set output limits for PID controllers
  positionPID.SetOutputLimits(motorMin, motorMax);
  speedPID.SetOutputLimits(motorMin, motorMax);

  // Set sample time for PID controllers in milliseconds
  positionPID.SetSampleTime(10);
  speedPID.SetSampleTime(10);

  // Enable automatic mode for PID controllers
  positionPID.SetMode(AUTOMATIC);
  speedPID.SetMode(AUTOMATIC);
}

void loop() {
  // Read the position and speed inputs from sensors or encoders
  positionInput = readPosition();
  speedInput = readSpeed();

  // Compute position PID control signal
  positionPID.Compute();

  // Compute speed PID control signal
  speedPID.Compute();

  // Set the motor speed based on the speed PID output
  if (digitalRead(Limit1) == HIGH){
    analogWrite(motorPin1, speedOutput);
    analogWrite(motorPin2,LOW);
  }
  else if(digitalRead(Limit2) == HIGH){
    analogWrite(motorPin2, speedOutput);
    analogWrite(motorPin1,LOW);
  }

}

// Function to read the position from a sensor or encoder
double readPosition() {
  // Implement your code here to read the position
  // and return the measured value
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
```

```
  delayMicroseconds(2);
  digitalWrite(trigPin, LOW);

  //Measure the duration of the pulse
  duration = pulseIn(echoPin, HIGH);

  //Calculate the distance in centimeters
  distance = duration * 0.034 / 2;
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");

  delay(500);
}

// Function to read the speed from a sensor or encoder
double readSpeed() {
  // Implement your code here to read the speed
  // and return the measured value
  if digitalRead(encoderPin) == HIGH){
    count++;
  }
  rotation = (count/pulses_per_turn)*60.0;
  Serial.println("RPM: ", rotation);
  delay(100);

}
```

VII. Conclusion

In this report, we explored the application of a dual PID control system for an elevator, focusing on position and speed control. The control system was designed to improve the elevator's performance, precision, and efficiency.

By employing a dual PID controller, we were able to address both position and speed control aspects of the elevator. The position controller utilized a PID algorithm to regulate the elevator's position accurately and efficiently. It adjusted the motor's output based on the error between the desired position and the actual position, considering proportional, integral, and derivative terms. This enabled precise and smooth positioning of the elevator at the desired floors.

In addition to position control, the speed controller further enhanced the elevator's performance. It maintained a consistent and controlled speed during elevator movements,

preventing abrupt changes and ensuring passenger comfort and safety. The dual PID speed controller incorporated the same PID algorithm, adapting the motor's output based on the speed error between the desired velocity and the actual velocity.

The use of a dual PID control system for elevator control offers several advantages. It provides accurate and responsive position control, resulting in precise floor-leveling and smooth door operations. Simultaneously, the speed controller ensures a comfortable and controlled elevator ride, minimizing jerks or sudden accelerations.

Through the analysis of step responses, impulse responses, root locus, and Bode plots, we gained insights into the system's behavior, stability, and performance. The step and impulse responses allowed us to evaluate the system's transient and steady-state characteristics, while the root locus and Bode plots provided information about stability, gain margins, and frequency response.

Overall, the implementation of a dual PID position and speed controller in the elevator control system demonstrated improved control accuracy, smooth operation, and enhanced passenger experience. By precisely regulating the position and speed, the control system ensures efficient and comfortable elevator transportation.

Further improvements could involve fine-tuning the PID gains based on specific elevator requirements and optimizing the control system's performance through advanced control techniques or adaptive algorithms. Additionally, incorporating safety features, fault detection mechanisms, and robust control methods would enhance the overall reliability and safety of the elevator control system.

In conclusion, the dual PID position and speed controller proved to be a valuable approach for elevator control, offering precise positioning, controlled speed, and an enhanced passenger experience. The successful implementation of this control system sets a foundation for further advancements in elevator technology, ensuring safe, efficient, and reliable vertical transportation in various applications.